SmartBus Astana

Project Integration, Testing and Improvement · Endterm

User goal: Help a passenger choose a less crowded departure: select a route, stop and time, compare predictions, then save the preferred option.

Scope: A working HTML/CSS/JavaScript prototype using simulated demand rules. No live CTS data, physical sensors or trained machine-learning model.

Team: Amangeldi Alikhan · Kerimbayev Akarys · Kartbay Alim

Group: [Group number — to be provided]

Same project as Assignment 4. Technical baseline: 1c2b3a7. The team supplied amanalikhan9020.github.io/smart-bus-v2/ as its A4 website; the exact submitted commit is not confirmed.

Functional progress & connected scenario

New functionality: compare three departure times and save the selected prediction.

Before: one prediction. After: the selected time, +30 and +60 minutes, with occupancy, passenger counts and a recommendation. Equal predictions are reported honestly.

Guest / login → route + stop + time → prediction → compare → select → save → Saved Trips → reload → retrieve the same choice.

Data flow: Inputs → shared calculation model → selected trip object → localStorage keyed by user ID → trip list and saved-data analytics.

Crossing midnight is labelled ‘Next day’. These are simulated demand comparisons, not real bus departure schedules.

The existing simulated route-demand chart remains a separate analysis. Not every page needs to be part of the same scenario.

Previous functionality: demonstrate live

19 automated browser tests passed (33.690 s, exit 0). Automated tests are not human usability tests.

Area	Working demonstration
Data storage	Save and retrieve after reload; filter and delete saved trips.
Validation	Reject empty/whitespace-only required input, name the field, prevent saving, then succeed after correction. Also check email format and duplicates.
Smart function	Input → occupancy and passengers. Bus 10, first stop: 18:00 → 56%; 05:00 → 10%. The same model powers comparison.
Data analysis	Average + maximum + a chart from saved predictions. Adding/deleting trips updates the results; empty data has a clear state.
Basic security	HTML input stays inert; wrong passwords are blocked. Salted PBKDF2, account separation and legacy migration/recovery. Local protection only.

User testing: two people outside the team

Both users must perform the same task in the working app; names and video are not required.

T1: Continue as guest. Choose Bus 10, the first stop and 18:00. Compare three options, save the least crowded, find it after reload, then report the saved average and maximum occupancy.

User	Task	Completion result	Difficulties / feedback
User 1	T1	[Not provided]	[Difficulties and feedback]
User 2	T1	[Not provided]	[Difficulties and feedback]

Record: completed independently / with help / not completed. Use the same app version and a fresh browser profile for each participant.

PENDING REAL TESTS. The previously supplied tester examples were explicitly illustrative; they are not recorded as actual results.

Two improvements based on real feedback

Required evidence: problem → change → before / after → repeat the affected task.

1. [Actual feedback] → [Change] → [Before / after] → [Repeated task and result]

2. [Actual feedback] → [Change] → [Before / after] → [Repeated task and result]

The team may repeat the affected task; inviting the users again is not required. One functional improvement may satisfy both progress and feedback criteria when the evidence supports it.

Implemented audit fixes: Security now opens; Save Trip reports success and duplicate errors; required fields are identified; plaintext passwords were replaced. All 19 browser checks passed.

These audit fixes cannot yet be claimed as two user-testing improvements. Actual feedback and the follow-up verification results still need to be collected.

Limitations, next steps & team contributions

Live presentation and demonstration: 8–10 minutes, followed by questions and functional checks.

Simulated occupancy, no live CTS or guaranteed arrival times. Local data has no device sync or server authentication. The browser owner can edit storage; PBKDF2 does not encrypt localStorage.

Next steps: Confirm the A3–A4 baseline; run two real user tests; implement two feedback-based improvements and repeat the tasks. Before Final: evaluate the model, consider verified data sources and server-side storage/access checks.

Contributions: Reported A4 work: Alikhan 33.3% — core logic/predictions; Akarys 33.3% — UI/charts; Alim 33.4% — validation/security/tests. Each member’s technical contribution AFTER A4: [confirm].

Every member must understand the complete scenario, explain relevant code/formulas/tests and answer individually. Slides or speaking alone are not enough; AI is allowed when the team understands the implementation.

Moodle: repository link OR source archive + final PDF (4–6 slides) + all names and group number. A separate report is not required. This draft still needs the missing evidence.